

K L Deemed to be University

Department of Computer Science and Engineering | 2026-2027, Odd Semester
Database Systems Engineering and Distributed Backend Development (25CS1302E)

Skilling Session 1: RDBMS Foundations

Name	Habeeb	Roll No.	2520090054
Course	25CS1302E	CO Mapping	CO1
Session	Skilling 1	Date	

Aim

To study the role of a DBMS over a file system, the RDBMS architecture and the three-schema (ANSI/SPARC) model, demonstrate logical and physical data independence, explore the system catalog and data dictionary in PostgreSQL, and audit the Task Hub frontend mock data into database entities (PBL).

Program

s1.sql (run with: psql -U postgres -f s1.sql)

```
-- Skilling 1: RDBMS Foundations - Task Hub project
-- Frontend audit (PBL): mock data found in the Task Hub React state
--   const users      = [{ id, name, email, role }]           -> entity USERS
--   const projects   = [{ id, name, ownerId, deadline }]     -> entity PROJECTS
--   const tasks      = [{ id, projectId, title, status,
--                        assigneeId, dueDate }]               -> entity TASKS
--   tasks[].status : 'TODO' | 'IN_PROGRESS' | 'DONE'         -> CHECK constraint
--   ownerId / projectId / assigneeId                         -> FOREIGN KEYS

DROP DATABASE IF EXISTS taskhub_db;
CREATE DATABASE taskhub_db;
\c taskhub_db

-- =====
-- CONCEPTUAL (logical) LEVEL: the complete community view of the data
-- =====

CREATE TABLE users (
  user_id   SERIAL PRIMARY KEY,
  full_name VARCHAR(100) NOT NULL,
  email     VARCHAR(150) NOT NULL UNIQUE,
  role      VARCHAR(10)  NOT NULL DEFAULT 'MEMBER' CHECK (role IN ('ADMIN', 'MEMBER'))
);

CREATE TABLE projects (
  project_id SERIAL PRIMARY KEY,
  name       VARCHAR(100) NOT NULL,
  owner_id   INT NOT NULL REFERENCES users(user_id),
  deadline   DATE
);

CREATE TABLE tasks (
  task_id     SERIAL PRIMARY KEY,
  project_id  INT NOT NULL REFERENCES projects(project_id) ON DELETE CASCADE,
  title       VARCHAR(200) NOT NULL,
  status      VARCHAR(12) NOT NULL DEFAULT 'TODO'
              CHECK (status IN ('TODO', 'IN_PROGRESS', 'DONE')),
  assignee_id INT REFERENCES users(user_id),
  due_date    DATE
);

INSERT INTO users (full_name, email, role) VALUES
('Habeeb',      'habeeb@taskhub.dev', 'ADMIN'),
('Ananya Rao',  'ananya@taskhub.dev', 'MEMBER'),
('Rahul Verma', 'rahul@taskhub.dev',  'MEMBER');

INSERT INTO projects (name, owner_id, deadline) VALUES
('Task Hub Backend',  1, '2026-11-30'),
('Task Hub Frontend', 2, '2026-12-15');

INSERT INTO tasks (project_id, title, status, assignee_id, due_date) VALUES
(1, 'Design ER diagram',      'DONE',      1, '2026-10-05'),
(1, 'Create PostgreSQL schema', 'IN_PROGRESS', 2, '2026-10-10'),
(1, 'Write FastAPI CRUD',      'TODO',      3, '2026-10-20');
```

```

(2, 'Replace mock data',      'TODO',      2, '2026-10-25');

-- =====
-- EXTERNAL (view) LEVEL: different users see different parts of the data
-- =====
-- View for a team member: only their own tasks, no e-mail addresses
CREATE VIEW member_task_view AS
SELECT u.full_name AS assignee, t.title, t.status, t.due_date
FROM   tasks t JOIN users u ON u.user_id = t.assignee_id;

-- View for a manager: progress summary per project
CREATE VIEW project_progress_view AS
SELECT p.name AS project,
       COUNT(t.task_id) AS total_tasks,
       COUNT(t.task_id) FILTER (WHERE t.status = 'DONE') AS done_tasks
FROM   projects p LEFT JOIN tasks t ON t.project_id = p.project_id
GROUP BY p.name;

SELECT * FROM member_task_view WHERE assignee = 'Ananya Rao';
SELECT * FROM project_progress_view ORDER BY project;

-- =====
-- INTERNAL (physical) LEVEL: how and where the data is stored
-- =====
SELECT relname AS table_name,
       pg_relation_filepath(oid::regclass) AS data_file,
       pg_size_pretty(pg_relation_size(oid)) AS size_on_disk
FROM   pg_class
WHERE  relname IN ('users', 'projects', 'tasks')
ORDER BY relname;

-- =====
-- LOGICAL DATA INDEPENDENCE: change the conceptual schema,
-- the external views keep working without any change
-- =====
ALTER TABLE tasks ADD COLUMN priority VARCHAR(6) DEFAULT 'MEDIUM';
SELECT * FROM member_task_view WHERE assignee = 'Ananya Rao';

-- =====
-- PHYSICAL DATA INDEPENDENCE: change the storage structure (add an index),
-- the same query text is answered without any change to the application
-- =====
SET enable_seqscan = off;  -- force the planner to show the index on a tiny table
EXPLAIN SELECT title FROM tasks WHERE assignee_id = 2;
CREATE INDEX idx_tasks_assignee ON tasks (assignee_id);
EXPLAIN SELECT title FROM tasks WHERE assignee_id = 2;
SELECT title FROM tasks WHERE assignee_id = 2;
RESET enable_seqscan;

-- =====
-- SYSTEM CATALOG / DATA DICTIONARY: metadata stored as tables
-- =====
-- information_schema (SQL-standard catalog)
SELECT table_name, table_type
FROM   information_schema.tables
WHERE  table_schema = 'public'
ORDER BY table_type, table_name;

SELECT column_name, data_type, is_nullable, column_default
FROM   information_schema.columns
WHERE  table_name = 'tasks'
ORDER BY ordinal_position;

-- pg_catalog (PostgreSQL native catalog): constraints and indexes
SELECT conname AS constraint_name,
       CASE contype WHEN 'p' THEN 'PRIMARY KEY' WHEN 'f' THEN 'FOREIGN KEY'
                   WHEN 'u' THEN 'UNIQUE'      WHEN 'c' THEN 'CHECK' END AS type,
       conrelid::regclass AS table_name
FROM   pg_constraint
WHERE  connamespace = 'public'::regnamespace
ORDER BY table_name, type;

SELECT indexname, tablename FROM pg_indexes WHERE schemaname = 'public' ORDER BY tablename;

-- psql meta-commands read the same catalog
\d
\dt
\dv

```

Output

```

DROP DATABASE IF EXISTS taskhub_db;
DROP DATABASE
CREATE DATABASE taskhub_db;
CREATE DATABASE
CREATE TABLE users (
    user_id SERIAL PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    role VARCHAR(10) NOT NULL DEFAULT 'MEMBER' CHECK (role IN ('ADMIN', 'MEMBER'))
);
CREATE TABLE
CREATE TABLE projects (
    project_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    owner_id INT NOT NULL REFERENCES users(user_id),
    deadline DATE
);
CREATE TABLE
CREATE TABLE tasks (
    task_id SERIAL PRIMARY KEY,
    project_id INT NOT NULL REFERENCES projects(project_id) ON DELETE CASCADE,
    title VARCHAR(200) NOT NULL,
    status VARCHAR(12) NOT NULL DEFAULT 'TODO'
    CHECK (status IN ('TODO', 'IN_PROGRESS', 'DONE')),
    assignee_id INT REFERENCES users(user_id),
    due_date DATE
);
CREATE TABLE
INSERT INTO users (full_name, email, role) VALUES
('Habeesb', 'habeesb@taskhub.dev', 'ADMIN'),
('Ananya Rao', 'ananya@taskhub.dev', 'MEMBER'),
('Rahul Verma', 'rahul@taskhub.dev', 'MEMBER');
INSERT 0 3
INSERT INTO projects (name, owner_id, deadline) VALUES
('Task Hub Backend', 1, '2026-11-30'),
('Task Hub Frontend', 2, '2026-12-15');
INSERT 0 2
INSERT INTO tasks (project_id, title, status, assignee_id, due_date) VALUES
(1, 'Design ER diagram', 'DONE', 1, '2026-10-05'),
(1, 'Create PostgreSQL schema', 'IN_PROGRESS', 2, '2026-10-10'),
(1, 'Write FastAPI CRUD', 'TODO', 3, '2026-10-20'),
(2, 'Replace mock data', 'TODO', 2, '2026-10-25');
INSERT 0 4
CREATE VIEW member_task_view AS
SELECT u.full_name AS assignee, t.title, t.status, t.due_date
FROM tasks t JOIN users u ON u.user_id = t.assignee_id;
CREATE VIEW
CREATE VIEW project_progress_view AS
SELECT p.name AS project,
COUNT(t.task_id) AS total_tasks,
COUNT(t.task_id) FILTER (WHERE t.status = 'DONE') AS done_tasks
FROM projects p LEFT JOIN tasks t ON t.project_id = p.project_id
GROUP BY p.name;
CREATE VIEW
SELECT * FROM member_task_view WHERE assignee = 'Ananya Rao';
  assignee | title | status | due_date
-----+-----+-----+-----
Ananya Rao | Create PostgreSQL schema | IN_PROGRESS | 2026-10-10
Ananya Rao | Replace mock data | TODO | 2026-10-25
(2 rows)

SELECT * FROM project_progress_view ORDER BY project;
  project | total_tasks | done_tasks
-----+-----+-----
Task Hub Backend | 3 | 1
Task Hub Frontend | 1 | 0
(2 rows)

SELECT relname AS table_name,
pg_relation_filepath(oid::regclass) AS data_file,
pg_size_pretty(pg_relation_size(oid)) AS size_on_disk
FROM pg_class
WHERE relname IN ('users', 'projects', 'tasks')
ORDER BY relname;
  table_name | data_file | size_on_disk
-----+-----+-----

```

```

projects | base/16769/16782 | 8192 bytes
tasks    | base/16769/16794 | 8192 bytes
users    | base/16769/16771 | 8192 bytes
(3 rows)

ALTER TABLE tasks ADD COLUMN priority VARCHAR(6) DEFAULT 'MEDIUM';
ALTER TABLE
SELECT * FROM member_task_view WHERE assignee = 'Ananya Rao';
  assignee |          title          | status   | due_date
-----+-----+-----+-----
Ananya Rao | Create PostgreSQL schema | IN_PROGRESS | 2026-10-10
Ananya Rao | Replace mock data         | TODO      | 2026-10-25
(2 rows)

SET enable_seqscan = off;
SET
EXPLAIN SELECT title FROM tasks WHERE assignee_id = 2;
               QUERY PLAN
-----
Seq Scan on tasks  (cost=10000000000.00..10000000011.88 rows=1 width=418)
  Filter: (assignee_id = 2)
(2 rows)

CREATE INDEX idx_tasks_assignee ON tasks (assignee_id);
CREATE INDEX
EXPLAIN SELECT title FROM tasks WHERE assignee_id = 2;
               QUERY PLAN
-----
Index Scan using idx_tasks_assignee on tasks  (cost=0.13..8.15 rows=1 width=418)
  Index Cond: (assignee_id = 2)
(2 rows)

SELECT  title FROM tasks WHERE assignee_id = 2;
      title
-----
Create PostgreSQL schema
Replace mock data
(2 rows)

RESET enable_seqscan;
RESET
SELECT table_name, table_type
FROM    information_schema.tables
WHERE   table_schema = 'public'
ORDER  BY table_type, table_name;
  table_name | table_type
-----+-----
projects    | BASE TABLE
tasks       | BASE TABLE
users       | BASE TABLE
member_task_view | VIEW
project_progress_view | VIEW
(5 rows)

SELECT column_name, data_type, is_nullable, column_default
FROM    information_schema.columns
WHERE   table_name = 'tasks'
ORDER  BY ordinal_position;
 column_name | data_type | is_nullable | column_default
-----+-----+-----+-----
task_id     | integer  | NO          | nextval('tasks_task_id_seq'::regclass)
project_id  | integer  | NO          |
title       | character varying | NO          |
status      | character varying | NO          | 'TODO'::character varying
assignee_id | integer  | YES         |
due_date    | date     | YES         |
priority    | character varying | YES         | 'MEDIUM'::character varying
(7 rows)

SELECT conname AS constraint_name,
       CASE contype WHEN 'p' THEN 'PRIMARY KEY' WHEN 'f' THEN 'FOREIGN KEY'
                   WHEN 'u' THEN 'UNIQUE'      WHEN 'c' THEN 'CHECK' END AS type,
       conrelid::regclass AS table_name
FROM    pg_constraint
WHERE   connamespace = 'public'::regnamespace
ORDER  BY table_name, type;
 constraint_name | type | table_name
-----+-----+-----

```

```

users_role_check      | CHECK      | users
users_pkey            | PRIMARY KEY | users
users_email_key       | UNIQUE     | users
projects_owner_id_fkey | FOREIGN KEY | projects
projects_pkey         | PRIMARY KEY | projects
tasks_status_check    | CHECK      | tasks
tasks_project_id_fkey | FOREIGN KEY | tasks
tasks_assignee_id_fkey | FOREIGN KEY | tasks
tasks_pkey            | PRIMARY KEY | tasks
(9 rows)

```

```
SELECT indexname, tablename FROM pg_indexes WHERE schemaname = 'public' ORDER BY tablename;
```

```

indexname      | tablename
-----+-----
projects_pkey   | projects
tasks_pkey      | tasks
idx_tasks_assignee | tasks
users_pkey      | users
users_email_key | users
(5 rows)

```

```

      List of relations
Schema | Name      | Type | Owner
-----+-----+-----+-----
public | projects  | table | postgres
public | tasks     | table | postgres
public | users     | table | postgres
(3 rows)

```

```

      List of relations
Schema | Name                  | Type | Owner
-----+-----+-----+-----
public | member_task_view      | view | postgres
public | project_progress_view | view | postgres
(2 rows)

```

Result

The Task Hub frontend state was mapped to the entities users, projects and tasks, and the three schema levels were implemented: base tables (conceptual), member and manager views (external) and data files on disk (internal). Adding the priority column left the views unchanged (logical data independence), and adding an index changed the plan from Seq Scan to Index Scan with the same query (physical data independence). Table, column, constraint and index metadata were read from information_schema and pg_catalog.